

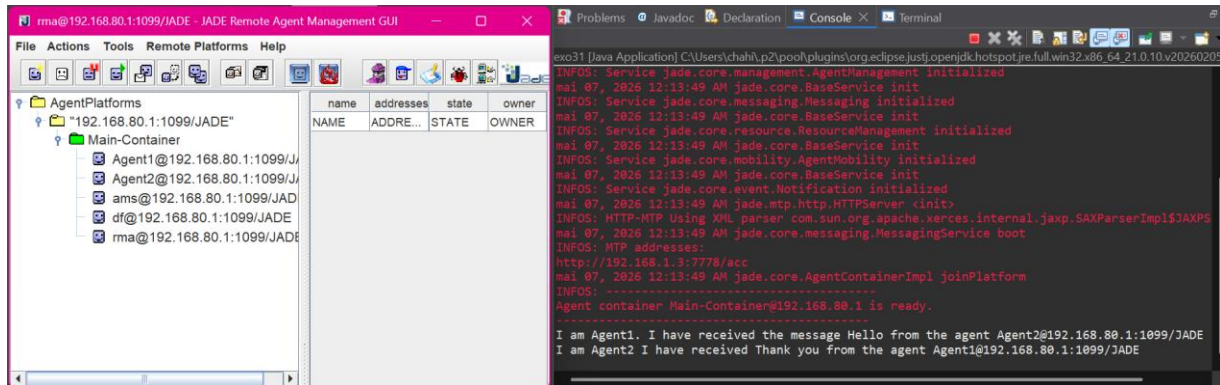
Chahinez Benaicha

LAB 3 RAPPORT

Advanced Communication between JADE Agents

Exercise 1: Point-to-point communication

- **Agent1:** waits for a message, receives "Hello", displays the message, then replies "Thank you" to the sender using an **ACLMessage** with performative **ACLMessage.INFORM**.
- **Agent2:** sends "Hello" to Agent1, waits for the reply, displays it.
- Implementation using CyclicBehaviour.



Exercise 2: Multiple containers

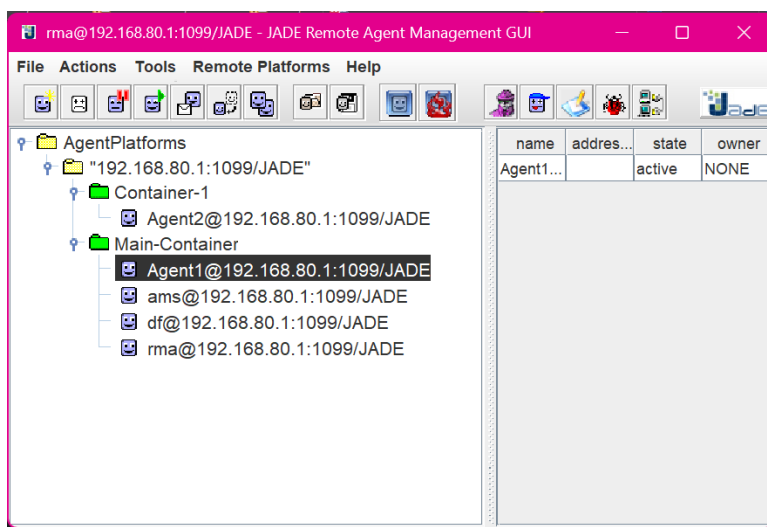
- Created a **main container** hosting the receiver agent (Agent2).

```
Runtime rt = Runtime.instance();
ProfileImpl pc = new ProfileImpl();
pc.setParameter(ProfileImpl.GUI, "true");
AgentContainer container = rt.createMainContainer(pc);
```

- Created a **simple container** hosting the sender agent (Agent1).

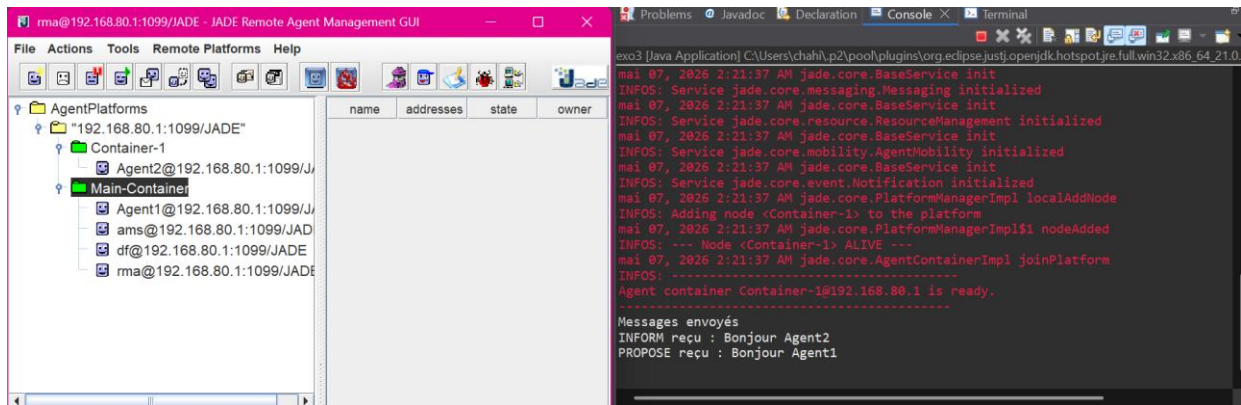
```
Runtime rt = Runtime.instance();
ProfileImpl pc = new ProfileImpl(false);
pc.setParameter(ProfileImpl.MAIN_HOST, "localhost");
AgentContainer container = rt.createAgentContainer(pc);
```

- Demonstrated that agents can communicate across different containers.



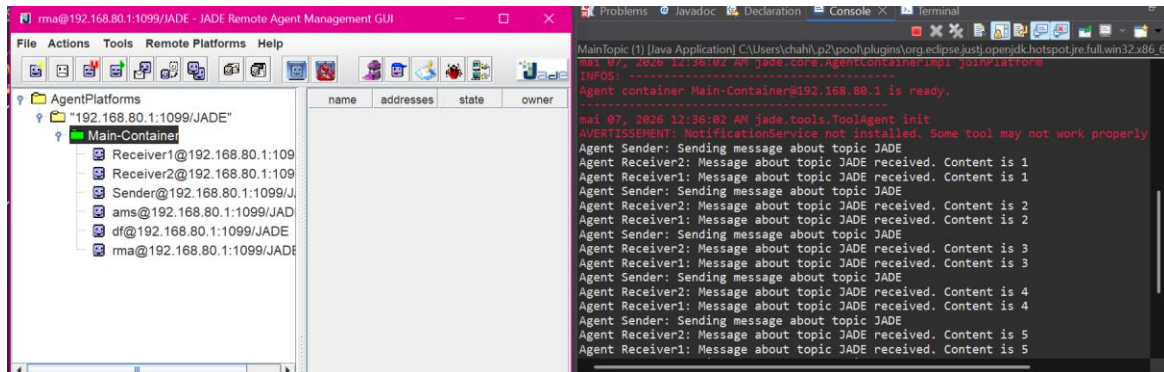
Exercise 3: Message filtering (MessageTemplate)

- The receiver agent uses “**MessageTemplate.MatchPerformative(ACLMessage.INFORM)**” to receive only INFORM-type messages.
- After receiving one, it changes its filter to accept only PROPOSE messages “**MessageTemplate.MatchPerformative(ACLMessage.PROPOSE)**”.
- This illustrates dynamic listening behaviour changes.



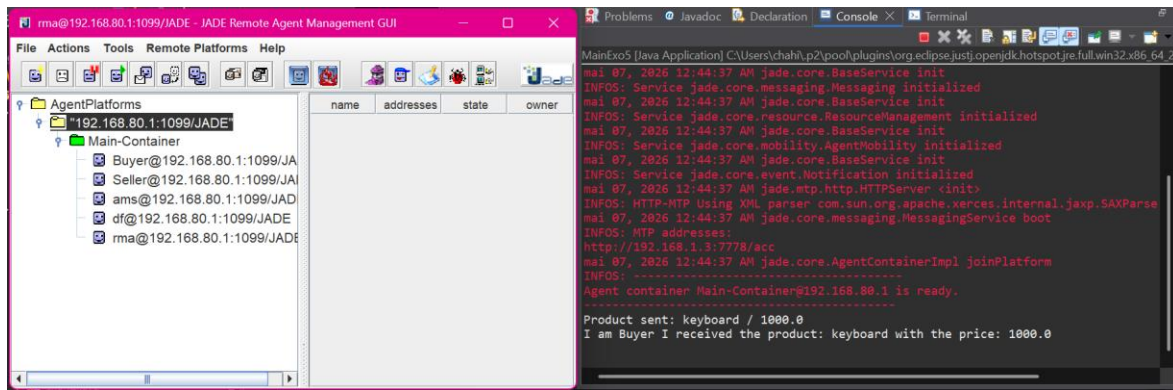
Exercise 4: Topic-based communication (broadcast)

- The sender agent creates a topic via “**createTopic("JADE")**”.
- Several receiver agents register to this topic via **registerOnTopic()**.
- The sender sends a message using **setTopic()**; all receivers get it.



Exercise 5: Object serialisation

- Defined a Product class implementing **java.io.Serializable** with attributes name (String) and price (double).
- **Seller:** creates a Product object, wraps it in an “**ACLMessage (performative ACLMessage.INFORM)**” using **setContentObject(product)**”.
- **Buyer:** receives the message, extracts the object via **getContentObject()**, casts it to Product, and displays its attributes.



Conclusion

This lab covered a wide range of agent communication needs: from simple point-to-point messaging to broadcasting, semantic filtering, and structured data exchange.

Jsfhbx d:kgdhs :g